Promise - obecanje je proxy(proxy su posrednicki objekti koji omogucavaju presretanje objekata) za vrednost koju ne moramo znati u vreme kreiranja. Moze biti u tri stanja:
1. pending: inicijalno stanje, niti ispunjeno niti odbijeno
2. fulfilled: operacija je uspesno zavrsena
3. rejected: operacija nije uspesno zavrsena
settled: fulfilled ili rejected
-Promise moze biti uspesan ili neuspesan jednom

Callback - funkcija kao parametar funkcije

Web storage je baza podataka u web citacu, cuva parove kljuc-vrednost, do 10MB u domenu, podrska u savremenim citacima
local storage: trajno cuva podatke
session storage: cuva podatke u toku sesije, gubi se po zatvaranju citaca

---

Event ima sledece atribute:
key: osobina koja je promenjena
newValue: nova vrednost
oldValue: stara vrednost
url: pun URL gde se dogadjaj desio
storageArea: localStorage ili sessionStorage objekat

Kako web citaci ucitavaju sadrzaj:
-Parsira HTML dokument redom. Elementi se dodaju u DOM kako se parsiraju. Kako se dodaju u DOM tako se
ucitavaju(slike,skriptovi,stilovi...)
-Citaci mogu citati vise resursa istovremeno
-Download eksternog JavaScript fajla ce blokirati druga dobavljanja jer bi skript mogao da promeni DOM ili preusmeri citac na drugu adresu. Citac nece poceti druga paralelna dobavljanja pre nego sto se skript preuzme, parsira i izvrsi.

Skript je zgodno staviti na sam kraj HTML fajla ispred </body>. To garantuje da ce citac parsirati ceo fajl i da je DOM spreman pre nego sto mu skript pristupi

Ako stavimo script na pocetak body pocece da se izvrsava pre nego sto je dokument u celosti parsiran(moze dovesti do greske)

CORS
Iz bezbednosnih razloga skriptovi su ograniceni samo na pristup podacima koji imaju isto poreklo.
Poreklo je kombinacija: URI sema: http ili https, hostname, port(default je 80, ali zavisi od browsera)

Access-Control-Request-Method: metoda pravog zahteva
Access-Control-Request-Headers: lista posebnih zaglavlja u zahtevu, razdvojenih zarezom

Access-Control-Allow-Origin: kao kod prostog zahteva
Access-Control-Allow-Methods: lista dozvoljenih HTTP metoda
Access-Control-Allow-Headers: lista dozvoljenih zaglavlja u zahtevu
Access-Control-Allow-Credentials: kao kod prostog zahteva
Access-Control-Max-Age: omogucava kesiranje ovog odgovora dati broj sekundi

---

var - promenljiva vidljiva svuda
Vidljive samo u bloku:
let: deklaracija promenljive
const: deklaracija konstante(ne mozes joj dodeliti drugu vrednost)

Repna rekurzija nece povecavati stek neograniceno, rekurzija je bezbedna. Ne dolazi do stack overflow

---

Cascade - mogucnost da pozovemo vise metoda prosledjujuci im isti objekat

---

Koncept modula: smestanje srodnih f-ja u poseban fajl, sprecavanje kolizije imena

---

Objekat je skup osobina(property), osobina je imenovani kontejner za vrednost

---

Closure je par koji cine funkcija i okruzenje za razresavanje slobodnih promenljivih

---

HTML dokument je DOM(document object model) stablo. Web citac predstavlja stranicu kao dokument objekat koji predstavlja koren stabla dokumenta.
Preko document korena mozemo: 1. promeniti stablo dokumenta,
2. napraviti novi HTML dokument iz pocetka ili
3. pristupiti i zameniti cvorove stabla(HTML elemente u DOM stablu)

Dogadjaj je objekat koji se salje kada se dogode akcije na stranici, najcesce kao posledica interakcije sa korisnikom
npr: click, mouseover, mouseout, keyup, load, focus, blur

---

rutiranje - omogucava navigaciju izmedju razlicitih prikaza, back i forward u web citacu, razmenu parametara izmedju kompnenti

---

Dependancy injection(zavisnost injekcija)
injekcija - mehanizam prosledjivanja reference na drugi objekat(npr prosledjivanje servisa komponenti)
dependancy - objekat treba da koristi drugi objekat, npr. komponenta poziva servis - komponenta je zavisna od servisa
zavisnost je objekat koji moze da se koristi iz drugog objekta

---

glubalni objekat - window
to su svi objekti koji se naprave, oni su svi u tom globalu

---

prototype - atribut koji pokazuje na osnovne funkcionalnosti, ima ga svaki objekat u js
ne postoji nasledjivanje nego se to radi preko prototype-a
new kreira objekat i poveze ga sa prototype-om

JSON web token - novi vid autentifikacije, proverava ima li fajlova vezanih za neki domen npr gmail

---

SPA - single page application

REST principi -> REST je servis koji stimulise upotrebu standarda: HTTP, URI, XML/HTML/JSON/GIF/JPEG
pojam resursa: svaki entitet na webu je resurs. Npr: web sajt, HTML strana,XML fajl, web servis, fizicki uredjaj
Svaki resurs je indentifikovan svojim URI-jem

RESTful servisi su stateless: stanje je iskljucivo na klijentu
transakcije u nadleznosti klijenta

Collection URI -- Element URI

GET izlista URI-je i eventualno druge podatke o elementima kolekcije--dobija reprezentaciju elementa kolekcije u obliku odgovarajuceg MIME tipa

POST kreira novi element kolekcije; URL novog elementa se vraca u odgovoru -- tretira dati element kao kolekciju i kreira novi element u njoj

PUT zameni celu kolekciju novom -- zameni element novim; ako ne postoji, kreira ga
DELETE uklanja celu kolekciju -- uklanja dati element kolekcije

---

HTTP na aplikativnom sloju u TCP/IP steku, protokol za komunikaciju na webu
connectionless: ne cuva konekciju
1. web citac inicira komunikaciju otvarajuci TCP vezu prema serveru
2. web citac salje zahtev serveru
3. server obradjuje zahtev i salje odgovor
4. server zatvara vezu...naredni zahtevi moraju ici kroz nove konekcije

media independent: bilo koji tip podataka (MIME-type) se moze prenositi sve dok klijent i server mogu da ih obrade
stateless: klijent i server su u vezi samo tokom jednog ciklusa zahtev/odgovor; ne cuva se stanje u komunikaciji

HTTP je klijent/server model
klijent salje zahtev: metoda,URI, verzija protokola, modifikatori zahteva, informacije o klijentu, opciono i sadrzaj
server salje odgovor: verzija protokola, kod rezultata, informacije o serveru, sadrzaj

Moguce metode u zahtevu -- metoda i opis u nastavku
GET za citanje podataka; ne bi trebalo da menja podatke
HEAD isto kao GET, ali vraca se samo prva linija i zaglavlja
POST za slanje (novih) podataka
PUT azurira resurs novim vrednostima(salje ceo resurs)
PATCH azurira resurs novim vrednostima(salje deo resursa)
DELETE uklanja resurs sa servera
CONNECT uspostavlja tunel prema serveru identifikovanim URI adresom
OPTIONS zahteva opis mogucih operacija za dati resurs
TRACE loopback test sa slanjem poruke

Status operacije -- kod i opis
1xx informativne poruke, od HTTP/1.1
2xx uspesno obradjen zahtev
    200 OK
    201 Created
    204 No Content
3xx redirekcija: usluga dostupna na drugom URI
    302 Moved Temporarily
    304 Not Modified
4xx greska na strani klijenta
    400 Bad Request
    401 Unauthorized
    403 Forbidden
    404 Not Found
5xx greska na strani servera
    500 Internal Server Error

Novosti u HTTP/2
multiplexing: vise asinhronih HTTP zahteva kroz jednu TCP konekciju, i odgovori se mogu primati asinhrono
svaki zahtev i odgovor imaju svoj stream id, podeljeni u delove(binarne) i svaki deo ima svoj frame id

stream je kolekcija frejmova

server push: vise odgovora na jedan zahtev
header compression: kompresija HTTP zaglavlja kao i sadrzaja
request prioritization: prioritet medju zahtevima poslatim na isti domen
binary protocol: HTTP/2 je binarni protokol, za razliku od HTTP/1.1

Primer: klijent salje zahtev za index.html; server proceni da ce klijent traziti i styles.css i script.js, i salje push promise da predupredi zahtev. Kaze u kojim strimovima ce biti odgovor za styles.css i script.js

---

Routing omogucuje navigaciju izmedju razlicitih prikaza, razmenu parametara izmedju komponenti koje upravljaju prikazima

Guards - interfejsi koji govore router-u da li da dozvoli prelazak na odredjenu rutu
rute zabranjujemo: korisnik nije autorizovan, nije autentifikovan(potreban login), podaci jos uvek nisu dostupni

---

Servisi su klase koje su dostupne razlicitim komponentama
njihova uloga je komunikacija sa serverskim delom aplikacije, omogucavaju komunikaciju izmedju komponenti

Komponente se bave view-related funkcionalnostima
servisi su zaduzeni za: dobavljanje podataka sa servera, validiranje korisnickih unosa, logovanje podataka na konzolu

---

Tokeni
Ako zelimo da sacuvamo tokene dobijene od nekih eksternih servisa i koristimo iz za autentifikaciju to mozemo uciniti tako sto ih stavimo u localstorage

Kako bismo se autentifikovali prilikom slanja zahteva serverima moram u zahtev umetnuti token koji nam je server izdao
jedan od nacina je da napravimo HTTP presretac koji presretne svaki zahtev i u njegov header stavi token
Drugi nacin za umetanje tokena je JWT modul koji presretanje radi automatski

---

Troslojna arhitektura
Kod treba da ima neku logicku strukturu, delimo ga u slojeve gde ce svaki sloj biti zaduzen za odredjeni deo posla
Slojevi mogu komunicirati ali samo preko apstrakcija(interfejsa) i to tako da se medjusobno ne preskacu i da nizi sloj ne vidi visi
Http zahtev -> sloj kontrolera -> sloj sevisa -> sloj podataka

Sloj kontrolera - ulazna tacka za spoljne zahteve ka aplikaciji, svaka metoda kontrolera se mapira na jedinstvenu URL putanja, uz to da mora biti definisan i HTTP glagol nad metodom(GET,POST,PUT...)
Sve operacije nad korisnicima staviti u UserController
Sloj servisa - biznis logika: sortiranje nizova, slanje mejlova ili bilo sta za namenu aplikacije
ne sme direktno komunicirati sa bazom i ne sme poznavati HTTP zahteve niti odgovore
ne sme da poziva metode kontrolera direktno
moze izvrsiti validaciju podataka i javiti kontrolerskom sloju ako je neka greska
Sloj podataka - jedini sme direktno da komunicira sa bazom, ali ne sme pozivati metodu nijednog sloja iznad trebao bi da enkapsulira upite ka bazi i da vraca modele kao odgovor

---

Dependancy inejction - injekcija zavisnosti
Predstavlja mehanizam za dodelu zavisnosti klasama koje ih koriste. Ako UserController komunicira sa UserService
servisom ne moze to ciniti direktno vec samo preko interfejsa
Sakrivanjem implementacija iza interfejsa postizemo slabiju povezanost izmedju implementacije i apstrakcije
i lakse testiranje metoda. Ciklusi: scoped, transient i singleton

---

Code-First: prvo kreiramo klase, pa na osnovu njih kreiramo bazu podataka
Database-First: iz postojece baze, kreiramo klase
Model-First: nekim alatom za modelovanje kreiramo model klasa i na osnovu njega generise klase i BP

Migracije c# kod koje ce Entity Framework pretvoriti u SQL skriptu

DTO - objekti koji nose podatke izmedju procesa ili slojeva aplikacije

---

Autentifikacija
Podatke o korisnicima mozemo cuvati u bazi podataka, a po unosu validnih kredencijala korisniku izdajemo json web token na osnovu kod se dalje autentifikuje pri pristupu nasim servisima
U token stavljamo sve podatke koji su nam bitni da znamo o korisniku, a koji nisu poverljivi jer token sam po sebi nije zasticen. Izdati token se potpisuje nasim privatnim kljucem ili lozinkom kako bi se sprecilo podmetanje vestackog tokena

U token mozemo staviti claimove(prava) koja korisnik ima. Mozemo staviti podatke o izdavaocu i primaocu tokena
i njegovo trajanje
Dobijeni token korisnik stavlja u HTTP zahteve prema nasem serveru i tako se autentifikuje
Nas server medjutim, mora verifikovati tokene iz svakog zahteva kako bi se uverili da se neko nije lazno predstavio ili podmetnuo token
Dodatno, kako bi validacija bila izvrsena mora se dodati autentifikacija u midlver

Authorize atribut za odredjivanje prava spram uloge inace ce moci svi koji imaju token

---

JWT io
ako zelimo da vidimo sadrzaj nekog tokena to mozemo uraditi na stranici jwt.io
kopiramo token i sa desne strane ce nam se prikazati njegov sadrzaj

---

CORS(cross origin resource sharing) je mehanizam kojim server specifira kojim domenima dozvoljava da ga kontaktiraju
Ovde recimo mozemo podesiti da iskljucimo frontend nase aplikacije moze slati zahteve zadnjoj strani

Web sadrzaj: staticki i dinamicki
Staticki: prethodno pripremljen web sadrzaj se salje kao odgovor klijentu(recimo HTML stranica)
na osnovu zahteva pronalazi se odgovarajuci fajl koji se vraca korisniku

nepostojanje zahtevanog fajla podrazumeva gresku 404 Not Found
Dinamicki: sadrzaj se generise nakon prijema zahteva

1. Proverava se da li postoji vec predefinisan sadrzaj(npr HTML)
2. Ukoliko sadrzaj postoji, vraca se kao odgovor
3. Ukoliko sadrzaj ne postoji, server pokusava da generise sadrzaj
4. Ako je sadrzaj uspesno generisan vraca se klijentu
5. Ako sadrzaj nije generisan 404 greska se vraca klijentu

Generisanje sadrzaja
Aplikativni server = web aplikacija - zaduzen za logiku
Programski jezici: c#, java, php, js
skladiste podataka(najcesce baza podataka)

Blockchain:
podaci orhanizova u blokove, podaci u narednom bloku zavise od prethodnih podataka, izrazita potreba u kriptovalutama
Distribuirano skladiste podataka- kopija se nalazi kod svih ucesnika u mrezi
Implementacija: GO, RUST
P2P komunikacija
Korisnici u okviru mreze odlucuju koji podatak moze da se sacuva u skladistu - kosenzus algoritmi

Ukljucivanje novog cvora, izmena podataka
Pregled podataka preko web-a, upisivanje podataka preko web-a (preko browsera na web server pa koji komunicira sa nekim node-om)

Potrebno je definisati potrebne funkcionalnosti, identifikovati ciljeve, revidirati razumevanje pre dizajna i implementacije

Use case dijagrami sluze da hvate:

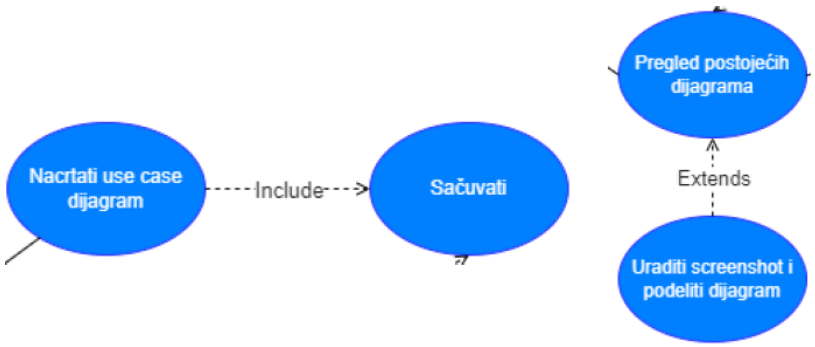
1. slucaj koriscenja(use case) - sta funkcionalnost treba da podrzi
   Postize cilj za ulogu. Koje funkcionalnost, sta svaka uloga mora da uradi, da li je ustanju to da uradi,sta su ulazi i izlazi iz sistema
2. Uloge u sistemu(actors) - definise se koje funkcije su kome dostupne
   neko ko ima interakciju sa sistemom predstavlja ulogu, a ne korisnika. Moze biti: uredja, softver, osoba. Komunikacija sa sistemom preko poruka(ulazno-izlazne)

Veze na dijagramu

- Includes: izdvaja zajednicke delove razlicitih slucajeva koriscenja
- Extends: opis slucaja koriscenja u posebnim slucajevima. Varijacija toka informacija(dodatna akcija koja moze, a ne mora da se izvrsi)

Modelovanje:

1. Identifikovati sve ucesnike i uveriti se da su obuhvacene sve funkcije koje koriste
2. Ako slucaj koriscenja zahteva ostale ucesnike, povezati i njih na dijagramu
3. Koristi extends da se opisu opciona ponasanja. Koristi include da se izdvoje zajednicki delovi



user story - agilne price

Oba sadrze: ko radi, sta radi, i kriterijum ispunjenosti

Misuse case - koristi se za testiranja kako bi dodatno obezbedili sistem i prepoznali malverzacije
(da li administrator sistema moze da obrise zalbu potrosaca na nestanak struje; da li zaposleni u distribuciji moze da se zali na nestanak struje)

Elektrodistribucija(DSO - distribution system operator)
Smart grid - zahtevi sistema:
Informacioni sistemi u modernim distribucijama se mogu grubo okarakterisati sledecim karakteristikama

1. nadziranje i upravljanje distributivnom mrezom
2. ocekivan rad u realnom ili blizu realnog vremena
3. detekcija i otklanjanje kvarova
4. planiranje razvoja distributivne mreze i pomoc pri izvodjenju radova
5. prikupljanje dodatnih podataka vezanim za distributivnu mrezu(npr multimedija)
6. Komunikacija sa ostalim IT sistemima

Veza sa webom:

Sve veci broj krajnjih potrosaca mora da ima svest o stanju ees
Elementi web-a neophodni za rad:
geolokacija ee elemenata u realnom vremenu(uber)
informacioni sistem za evidenciju i dokumente za rad
RBAC : sistem od vise razlicitih uloga korisnika(zaposleni u DSO, potrosaci...)
Video streaming, AI, twitter migracije...

SCADA - prikupljanje telemetrije i upisivanje vrednosti(upravljanje)
OMS - detekcija, analiza, upravljanje neplanski kvarovima
upravljanje ispadima u uslovima vremenskih nepogoda
ADMS - sveobuhvatan nadzor, estimacije stanja na nivou cele mreze
Proracun niza korektivnih i preventivnih upravljackih akcija optimizacija ees
GIS - mapa, konektivnost mreze, osnovna vidljivost u ees mrezi
CIS - personalni podaci individualnih potrosaca, novi prikljucak za struju - novi upis u CIS bazu, modul za naplatu elektricne energije
EAM - podaci o svakom individualnom objektu u ees mrezi, sluze za upravljanje radovima
MDM - akvizicija i skladistenje podataka o individualnoj potrosnji energije, obracun za naplatu
AMI - daljinsko ocitavanje potrosnje elektricne energije, status napajanja individualnog potrosaca, a sve cesce i daljinsko ocitavanje vrednosti napona na mestu potrosaca, pametna brojica
WMS - zavodjenje i izvestavanje o svim planiranim radovima na distributivnoj mrezi
radovi ne moraju rezultovati prekidom napajanja, npr. provera postrojenja, instalacija brojila, radovi izgradnje izamena brojila
WFM - ekipe koje rade u polju, platni spisak ekipe u polju, vrednost sistema je u optimizaciji planiranja rada ekipa i njihovih troskova

Osnovni izazovi:
- Obrada velike kolicine podataka
- Near-realtime odgovor sistema
- Pouzdanost podataka, security
- Robusnost sistema -> distribuirane transakcije, faioverm siste switch

---

Web server

Hardware
- Hostuje web server software
- Povezan je na internet(statick IP)
- Skladisti web sadrzaj(HTML i js fajlove, css,slike...)

Software
- Obezbedjuje klijentu (npr browser-Chrome) da pristupi web sadrzaju
- Komunikacija na nivo zahtev/odgovor
- IIS, Apache...

Osnovni zadaci - software:
Koristi odgovarajuci protokol za komunikaciju (najcesci HTTP/HTTPS)
Osluskuje zahteve na odgovarajucem portu(HTTO: port 80, HTTPS port 443)
Salje zahtevani resurs kao odgovor

RESTful servisi
- Klijent server
- Bez stanja(stateless)
- Racunati na kesiranje(cacheable)
- Jedinstveni interfejs
- Slojeviti sistem
- Kod na zahtev

Web1
Korisnici mogu samo da konzumiraju podatke(dobiju pregled informacija)
Nije omoguceno editovanje podataka od strane korisnika
Vlasnik web sajta je ujedno i vlasnik sadrzaja
Osnovni primer su staticke HTML stranice koje se dobiju od strane web servera

Web2
Internet postaje interaktivniji - dvosmerna razmena informacija
Korisniku je omoguceno da menja i ucestvuje u izgradnji web sadrzaja: socijalne mreze, blogovi, wikipedia, izrada use case dijagrama
Kompanije skupljaju podatke o nama kako bi na osnovu toga odredili kakav sadrzaj da nam prikazu i zadrze nas duze na svojim stranicama
Vlasnik web sajta i dalje ostaje vlasnik sadrzaja
Targeted advertising - kompanije na osnovu skupljenih podataka znaju koje oglase da nam prikazu
Interoperabilnost izmedju sajtova postaje vazna: logovanje preko Google-a, facebook-a, povlacenje sadrzaja sa drugih web sajtova
Pojavila se znacajnija kolicina informacija za obradu - SPA

Web3
Tezi da oslobodi internet decentralizacije: verovatno upotrebom blockchain tehnologije

Ne postoji jedinstvena tacka kontrole, vec vraca kontrolu korisnicima
Vecina u okviru mreze odlucuje koji podataka je tacan

---------- Kroz pricu nesigurno sve

Session cookie - koji god zahtev da posaljes uvek ti se vraca cookie
Server izdaje cookie, Cookie cuva podatke o klijentu na serverskoj strani
JWT token sam cuva podatke, ne cuvaju se na serveru, JWT se sifruje privatnim kljucem servera
HTTP Mine - ?

Claimove i JWT pita sigurno
JWT je objekat koji koristis da saljes podatke
Claim su ti podaci?
JWT - cookies razlika
razlika JWT - sesija
AccesControlAllowOrigin - CORS

Sa ispita(neki odgovori su gore)

JUN

1. sta je 401/403 HTTP zahtev
2. jwt vs session
3. jwt claim
4. staticki vs dinamicki web
5. osnovni izazovi smart grida
6. JS zadatak

Imas funkciju koja vraca funkciju pa sta ce biti ispisi
U sustini kontrola scope-ova funkcija i memorije
da li prvo js trazi varijablu unutar svog scopa ili global scopa

```
var x = 100;
var f = function(){
  var x = 200;
  return function(){
    x += x;
    return x;
  }
}
var g = f();
console.log(g());
console.log(x+x);
console.log(g());
```

Resenje:
400
200
800

JUL

1. 0bjasniti SCADA(Supervisory Control and Data Acquisition) podsistem u okviru Smart Grid-a
2. Cemu sluzi nmp install komanda u okviru node okruzenja?

Instalacija npm paketa postujuci redosled prioriteta ukoliko on postoji. Podrazumevano instalira sve module iz package.json.

node package manager for the Node JavaScript platform. It puts modules in place so that node can find them, and manages dependency conflicts intelligently.

3. Koje vrste web storage-a postoje? Navesti primer za bar jednu od njih, u kojim situacijama bi se mogla koristiti

4. Razlike HTTP/1.1 i HTTP/2.0 protokola (sve pisu na jednom slajdu)

- **HTTP/1.0** koristi novu TCP konekciju za svaki ciklus zahtev/odgovor
- **HTTP/1.1** može da koristi istu TCP konekciju za više ciklusa zahtev/odgovor
- ali je protokol i dalje connectionless i stateless
- uvedeno zbog unapređenja performansi
    - jedna stranica sa 20 slika bi zahtevala 21 TCP konekciju po HTTP/1.0
    - jedna stranica sa 20 slika, 5 CSS fajlova i 10 JavaScript fajlova...
- **HTTP/2.0** novine radi unapređenja performansi

HTTP 2.0

- smanjenje kašnjenja
- smanjenje broja otvorenih TCP konekcija
- bolja bezbednost na webu
- kompatibilnost sa HTTP/1.1 klijentima i serverima
- može se koristiti gde i HTTP/1.1

5. Objsniti prednost REST servisa(zasto su danas toliko rasprostranjeni) -- termin 10 predavanja

1. Raznovrsnost klijenata
2. Prilagodljivost
3. Pristup informacijama koje ne mozemo posedovati
4. REST je odlican za monetizaciju podataka

6. Razmotriti sledeci kod i navesti koje metode je moguce pozvati nad objektima: userOne, userTwo, adminOne, adminTwo:

```
function User(email, name){
  this.email = email;
  this.name = name;
  this.online = false;
}

User.prototype.login = function(){
  this.online = true;
  console.log(this.email, 'has logged in');
};

var userOne = new User('ryu@xxx.com', 'Ryu');

function Admin(email, name){
  User.call(this, email, name);
}

Admin.prototype = Object.create(User.prototype);

Admin.prototype.deleteUser = function(){
  console.log('Admin deletes a user');
};

User.prototype.additionalUserFunc = function(){
  console.log('Additional user function');
};

var userTwo = new User('yoshi@xxx.com', 'Yoshi');
var adminOne = new Admin('shaun@xxx.com', 'Shaun');

Admin.prototype.additionalAdminFunc = function(){
  console.log('Additional admin function');
};

var adminTwo = new Admin('mario@xxx.com', 'Mario');
```

main.js

Run

```javascript
    }
Admin.prototype.additionalAdminFunc = function(){
console.log('Additional admin function');
}
var userTwo = new User('yoshi@xxx.com', 'Yoshi');

var adminOne= new Admin('shaun@xxx.com', 'shaun');

User.prototype.additionalUserFunc = function(){
    console.log('Additional user function');
    }

var adminTwo = new Admin('mario@xxx.com', 'Mario');

userOne.login();
//userOne.deleteUser();
//userOne.additionalAdminFunc();
userOne.additionalUserFunc();

userTwo.login();
//userTwo.deleteUser();
//userTwo.additionalAdminFunc();
userTwo.additionalUserFunc();

adminOne.login();
adminOne.deleteUser();
adminOne.additionalAdminFunc();
adminOne.additionalUserFunc();

adminTwo.login();
adminTwo.deleteUser();
adminTwo.additionalAdminFunc();
adminTwo.additionalUserFunc();
```

Output

```
node /tmp/OgonI8Ep0e.js
ryu@xex.com has logged in
Additional user function
yoshi@xxx.com has logged in
Additional user function
shaun@xxx.com has logged in
Admin deletes a user
Additional admin function
Additional user function
mario@xxx.com has logged in
Admin deletes a user
Additional admin function
Additional user function
```